

Baseline Reproduction Protocol & Report

Reproducing 3D Gaussian Splatting and Mip-Splatting on free compute — verified against repository source before deployment. What will be reproduced, what provably cannot be, what counts as success, and the three measurements that turn the reproduction from a prerequisite into the project's first experiment.

Candidate	Rickarya Das · 23EE30019	Companion to	Radiance Fields BTP Handbook v4
Project	Sampling-Geometry Band-Limits for Radiance Fields	Compute	Free tier – T4 16 GB, quota-limited
Version	v3 · 7 September 2026 · deployment-ready	Status	All flags resolved · protocol frozen · results pending

00 Provenance, and what v2 changed

v1 was written from the papers and the repository READMEs. v2 added a pass over the benchmark scripts. **v3 closes every remaining open flag by reading the loaders, the model and the evaluation code** — `scene/__init__.py`, `scene/dataset_readers.py`, `scene/gaussian_model.py`, `render.py`, `metrics.py`, `arguments/__init__.py`, `convert_blender_data.py`. Sixteen findings follow in §01. Two of them **reverse a v2 decision**, and one exposes a step the shipped pipeline cannot perform at all. After §01 there are no undecided branches: exactly two facts remain unknown, both readable off a screen in the first session.

CHIP	MEANS
3DGS	Kerbl, Kopanas, Leimkühler & Drettakis, ACM TOG 42(4), 2023 — figures as recorded in Handbook §02, checked against the paper.
MIP·PAPER	Yu, Chen, Huang, Sattler & Geiger, <i>Mip-Splatting</i> , CVPR 2024 — tables read from arXiv:2311.16493.
GS·SRC	graphdeco-inria/gaussian-splatting — README.md, <code>full_eval.py</code> , <code>metrics.py</code> on main, read 6–7 Sep 2026.
MIP·SRC	autonomousvision/mip-splatting — README.md and the complete contents of <code>scripts/</code> on main, read 7 Sep 2026.
NB	nerfbaselines/nerfbaselines README and the published Mip-NeRF 360 leaderboard, read 6 Sep 2026.
HB	Radiance Fields BTP Handbook v4, 30 Aug 2026.
EST	An estimate by this report, basis stated. Replaced by measurement at gate G1 .
UNVERIFIED	Load-bearing but not confirmable from a primary source here. On the pre-flight checklist, §13.

STATUS

Every results column is still empty, and the figures in Part I are drawn from *published* numbers — they are the target, not a claim. They regenerate as our own figures from `results/runs.csv` by one command once runs land, with the published curve retained as a faint dashed reference. The protocol, acceptance criteria, budget and gates are frozen *before* any run, so a disappointing number later cannot be rationalised into a success.

01 Resolutions — every open flag closed

Each entry names what was checked, in which file, and what it settles. **BREAKING** marks a finding that would have silently corrupted the reproduction; **REVERSAL** marks a v2 decision this pass overturned.

1.1 · The architecture decision, now closed

ID	FINDING	WHAT IT SETTLES
F1	BREAKING The Mip-Splatting repository ships no 3DGS baseline. <code>scripts/</code> holds exactly <code>fused_ply.py</code> , <code>run_mipnerf360.py</code> , <code>run_mipnerf360_stmt.py</code> , <code>run_nerf_synthetic_mmt.py</code> , <code>run_nerf_synthetic_stmt.py</code> ; all pass <code>--kernel_size 0.1</code> .	Every table in both papers has a 3DGS column you must build. Half the compute budget is the arm nobody ships.
F2	REVERSAL BREAKING The original 3DGS repository cannot read the multi-scale data at all. <code>convert_blender_data.py</code> writes <code>images_train/</code> <code>images_val/</code> <code>images_test/</code> and <code>metadata.json</code> — the mip-NeRF multiscale format, <i>not</i> <code>transforms_train.json</code> . <code>scene/__init__.py</code> dispatches on <code>filename</code> , and <code>sceneLoadTypeCallbacks</code> in Mip-Splatting has a third entry, "Multi-scale": <code>readMultiScaleNerfSyntheticInfo</code> , which 3DGS lacks.	v2 preferred arm B-i, the original repo. That is impossible, not merely risky — it falls through every branch and dies with <code>NameError: scene_info</code> . It could not work in principle either: <code>readMultiScale</code> sets <code>fovx</code> from <code>meta["focal"][idx]</code> <i>per frame</i> , where the Blender reader takes one global <code>camera_angle_x</code> . Gate G0 is closed: everything runs in the Mip-Splatting codebase, both arms.
F3	The 3D filter enters through exactly two properties. <code>get_scaling_with_3D_filter =</code> <code>√(s²+f²)</code> and <code>get_opacity_with_3D_filter</code> with <code>coef = sqrt(det1/det2)</code> . Setting <code>filter_3D = 0</code> makes both the identity.	Arm B is a two-line, provably exact change plus <code>--kernel_size 0.3</code> : not an approximation of 3DGS but a reduction to it. §06 gives the diff verbatim. It doubles as the exact-reduction unit test Handbook §09 already wanted.

F4	STMT versus MTMT is one named flag. <code>ModelParams.load_allres</code> defaults <code>False</code> ; <code>readMultiScale(..., only_highres)</code> skips every file not ending <code>d0.png</code> for the training split.	No bespoke data preparation per protocol. R2 is R1 plus <code>--load_allres</code> .
-----------	---	--

WHAT THE SINGLE CODEBASE BUYS

Both arms share one dataloader, one `metrics.py` and one LPIPS implementation, so a difference between them is the method rather than the plumbing — a *stronger* comparison than cross-repository, not a weaker one. It also deletes an entire risk class: the post-publication 3DGS options (`--antialiasing`, sparse-Adam, exposure, depth) can no longer leak in, because that repository is never on the critical path. Run it after R3 on the COLMAP scenes, which it *can* read, purely as a cross-check.

1.2 · The step the shipped pipeline cannot perform

ID	FINDING	WHAT IT SETTLES
F5	BREAKING <code>metrics.py</code> returns one number where the paper reports four. <code>render_set</code> writes <code>test_preds_{r} / gt_{r}</code> where <code>{r} = dataset.resolution</code> — the <code>-r</code> flag, <i>not</i> the per-image scale — so with the default <code>-r -1</code> all four scales land in one directory, and <code>metrics.py</code> averages the lot.	Without a post-processing step there is no Table 2 reproduction , only a single blended average. Resolved by <code>tools/split_by_scale.py</code> , supplied with this report: it groups <code>per_view.json</code> by scale using the order-preserving map <code>render {idx:05d}.png ↔ metadata.json["test"] ["file_path"][idx]</code> , whose <code>_d{j}</code> suffix is the scale. No re-rendering, no resampling. Re-running <code>render.py</code> at <code>-r 1 2 4 8</code> would resample and change the experiment.
F6	metrics.py hides its failures. Each scene is wrapped in a bare <code>except</code> : that prints "Unable to compute metrics for model" and continues.	A hard failure looks like a warning and an empty column looks like a bad result. Assert <code>results.json</code> exists and is non-empty after every metrics call.

1.3 · Evaluation and environment

ID	FINDING	WHAT IT SETTLES
F7	REVERSAL The Blender test split is not every-8th. It is the dataset's own test split; <code>llffhold=8</code> appears only in <code>readColmapSceneInfo</code> .	v2's §4.4 stated "every 8th photo held out" without qualification. Corrected: every-8th applies to the COLMAP scenes — Mip-NeRF 360, Tanks & Temples, Deep Blending — only.
F8	BREAKING LPIPS is VGG in both repositories, though via different implementations: Mip-Splatting uses the official <code>lpips</code> package (<code>lpips.LPIPS(net='vgg')</code>), 3DGS a vendored <code>lpipsPyTorch</code> .	AlexNet — the common default — is not comparable to either paper, and the discrepancy reads as a method effect. The single-codebase decision makes both arms share one implementation, so this is settled rather than merely pinned. Requires <code>pip install lpips</code> .
F9	BREAKING P100 is ineligible. 3DGS requires Compute Capability 7.0+; T4 is 7.5, P100 is 6.0.	Kaggle offers both. Select T4 and assert <code>get_device_capability() == (7, 5)</code> at start-up, before a build silently fails.

F10	filter_3D, exactly. <code>distance / focal_length * (0.2 ** 0.5)</code> , shape <code>[N,1]</code> ; <code>distance</code> is the minimum clamped <code>z</code> over cameras that see the point (<code>z > 0.2</code> , within a 1.15× screen margin); <code>focal_length</code> is the maximum <code>camera.focal_x</code> , taken independently. Gaussians no camera sees are assigned <code>distance[valid].max()</code> .	Confirms the Handbook's reading of the floor, and gives I-1 its exact target. The unseen-Gaussian branch must be excluded or flagged in the histogram or it forms a spurious right tail that would be misread as the floor binding.
F11	Free 2× parallelism, already implemented. The benchmark scripts ship a <code>GPUUtil dispatcher</code> — <code>ThreadPoolExecutor(max_workers=8)</code> , one scene per free GPU.	Kaggle's 2×T4 is used with no code change (<code>pip install gputil</code>). Roughly halves the elapsed quota for R1 and R2; GPU-hours are unchanged.
F12	Defaults confirmed. <code>_kernel_size = 0.1</code> (so the shipped script passes the default explicitly; 3DGS's fixed dilation is 0.3); <code>iterations = 30 000</code> ; <code>densification_interval = 100</code> ; Blender flags <code>--eval --white_background</code> ; scenes <code>ship drums ficus hotdog lego materials mic chair</code> ; images named <code>{:03d}_d_{j}.png</code> with focal <code>f / 2**j</code> .	Fixes every R1/R2 launch argument. Nothing about the run configuration is now inferred from paper prose.
F13	Real-scene settings. <code>outdoor -i images_4</code> , <code>indoor -i images_2</code> , T&T and Deep Blending at default; <code>--disable_viewer --quiet --eval --test_iterations -1</code> ; renders at both 7 000 and 30 000.	R3 is fully specified, and the 7 K row of the 3DGS table is free.
F14	nerfbaselines cannot run the primary target — it exposes standard dataset protocols only; STMT and MTMT are Mip-Splatting's own.	Demoted from v1's primary toolchain to an independent cross-check behind §4.3. With F2, the critical path uses neither it nor the 3DGS repository.
F15	Free-tier limits SECONDARY : 30 h weekly GPU quota, 12 h maximum GPU session, accelerators T4 ×2 or P100, ~20 GB auto-saved <code>/kaggle/working</code> .	With F11's dispatcher, R1 is roughly 5–8 h wall-clock — one session . Read the true values off the notebook sidebar in session 1; the ladder does not depend on them, only the calendar does.
F16	Disk pressure. Eight scenes × four scales × ~200 test views × 2 (render + ground truth) is roughly 2.5 GB of PNGs per arm, against ~20 GB of session storage.	Fits, but only with hygiene: once a scene's metrics succeed, delete <code>test_preds_*</code> and <code>gt_*</code> , keeping <code>results.json</code> , <code>per_view.json</code> , the checkpoint and a fixed set of qualitative views.

EVERYTHING THAT IS STILL UNKNOWN

Two facts, neither readable from source, neither on the critical path. **(i)** Kaggle's true quota and session cap — read them off the sidebar in session 1 (F15). **(ii)** Whether `counter` and `drjohnson` fit 16 GB — unknowable without running, and R3's exit rule already covers it: record the OOM, do not work around it. There is nothing else left to decide at deployment.

02 What this report is for

The BTP replaces Mip-Splatting's scalar 3D smoothing filter with an anisotropic band-limit derived from the accumulated sampling Fisher information, floored at Mip-Splatting's own value `HB`. The method is defined by its relationship to two baselines, so it cannot be evaluated — arguably cannot be honestly described — until both run under my hand, on my hardware, with my evaluation code.

Three things follow, and they are why this is a report rather than a task:

1. **The parity prediction is unfalsifiable without a measured spread.** Handbook §07 predicts parity on well-conditioned captures. Parity is a statement about a distribution; until the seed-to-seed standard deviation of my own 3DGS runs is known, parity cannot be distinguished from a small real loss.
2. **The floor may already dominate.** B1's estimation term appears only where $s^2/\lambda_i > f_k^2$. If the inherited floor is large across essentially all Gaussians on real captures, B1 collapses to the baseline everywhere and there is no project. This is answerable from an *instrumented baseline run* at a cost of hours.
3. **Λ can be profiled on baseline checkpoints.** The accumulated Fisher information is a function of the cameras and the Gaussian positions; it requires no change to the optimiser and no CUDA.

THE SENTENCE FOR YOUR SUPERVISOR

"The reproduction is not a warm-up. Instrumenting the two baselines produces the error bar that makes my parity prediction falsifiable, and it answers — for the cost of two GPU-days — the single question that can kill the project: whether Mip-Splatting's floor already binds everywhere."

03 What "reproduced" is allowed to mean

Free compute is 16 GB. The 3DGS repository states "24 GB VRAM (to train to paper evaluation quality)" `GS · SRC`. Numerical identity with the published tables is therefore unavailable for the largest scenes, and pretending otherwise would be the first indefensible claim in the thesis. Three levels, and every result states which it claims.

LEVEL	CLAIM	CRITERION	WHERE ATTAINABLE
L1	Numerical reproduction	Mean PSNR within 0.20 dB , SSIM within 0.005 , LPIPS (VGG) within 0.010 of the published dataset average, at the paper's own resolution, iteration count and split.	Blender synthetic only. <code>EST</code>
L2	Phenomenon reproduction	The <i>effect</i> the paper claims reappears with correct sign, ordering and order of magnitude, under a stated deviation in resolution or scene subset. Absolute values are not compared.	Everywhere, including Mip-NeRF 360 outdoor at reduced resolution.
L3	Harness parity	My own re-run baseline is the comparison point for all later method results. Published numbers are cited but never used as the control.	Everywhere. This is the level that governs the thesis tables.

L3 is not a concession — it is what 3DGS does. Its Table 1 marks the Mip-NeRF 360 row as copied from the original publication and reports its own re-run separately (PSNR 27.58 vs the copied 27.69) `HB`. An independent prior on how

much implementation identity is worth appears in §04.5: faithful splatting implementations spread across roughly 0.6 dB on the same benchmark, which is three times the L1 tolerance.

THE CLAIM THIS REPORT WILL MAKE

L1 on the Blender synthetic set; L2 on everything real; L3 throughout. Every table cell carries GPU model, resolution flag, iteration count, seed, LPIPS backbone, and both repository commits — the discipline Handbook §12 already requires, extended by F8 and F2.

PART I · THE REFERENCE NUMBERS

04 Published results to be reproduced

These are the targets, transcribed against a fixed sourced record. The figures plot the same numbers; they are the paper's results, not ours.

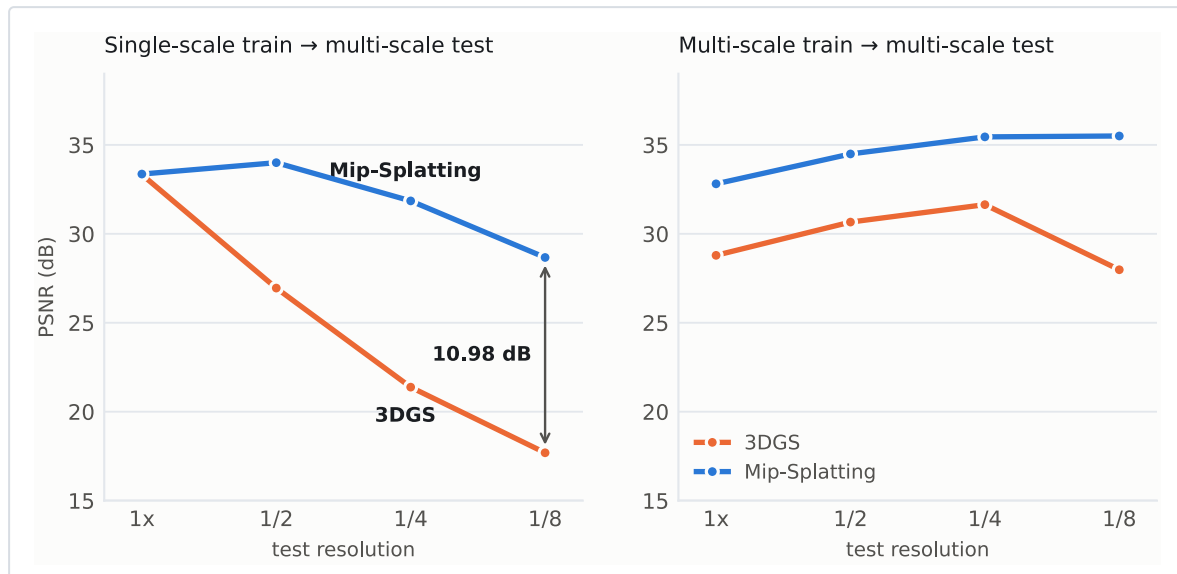


Figure 1 — the phenomenon the project inherits. Blender synthetic, mean PSNR over eight scenes. **Left:** trained at one scale, tested at four. 3DGS loses 15.64 dB between full resolution and $\frac{1}{8}$ because nothing limits primitive size to what the training views can resolve; Mip-Splatting loses 4.69 dB. **Right:** when the training set already contains all four scales, 3DGS recovers but still trails at every scale. The left panel is the R1 target and the reason it is first: an 11 dB effect survives any hardware difference between an A6000 and a T4. Source [MIP · PAPER](#) Tables 1–2.

4.1 · 3DGS, Table 1 dataset averages 3DGS

METHOD	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	TRAIN	FPS	MEMORY
MIP-NeRF 360 · 9 SCENES						
Plenoxels	0.626	23.08	0.463	25m49s	6.79	2.1 GB
INGP-Base	0.671	25.30	0.371	5m37s	11.7	13 MB
INGP-Big	0.699	25.59	0.331	7m30s	9.43	48 MB
Mip-NeRF 360 $\uparrow$	0.792	27.69	0.237	48 h	0.06	8.6 MB
3DGS — 7 K	0.770	25.60	0.279	6m25s	160	523 MB

3DGS — 30 K	0.815	27.21	0.214	41m33s	134	734 MB
TANKS & TEMPLES · DEEP BLENDING — MIP-NeRF 360 / 3DGS-30K						
T&T (truck, train)	0.759 / 0.841	22.22 / 23.14	0.257 / 0.183	48 h / 26m54s	0.14 / 154	8.6 MB / 411 MB
DB (drjohnson, playroom)	0.901 / 0.903	29.40 / 29.41	0.245 / 0.243	48 h / 36m2s	0.09 / 137	8.6 MB / 676 MB
BLENDER SYNTHETIC · 8 SCENES						
3DGS — 30 K, mean PSNR	—	33.32	—	—	180–300	≈3.8 MB

† Copied by the 3DGS authors from the original Mip-NeRF 360 publication; their own re-run averaged 27.58 / 0.790 / 0.240. Timing is not like-for-like — the 48 h is a single-GPU equivalent of a 12 h four-A100 run, against 3DGS on one A6000.

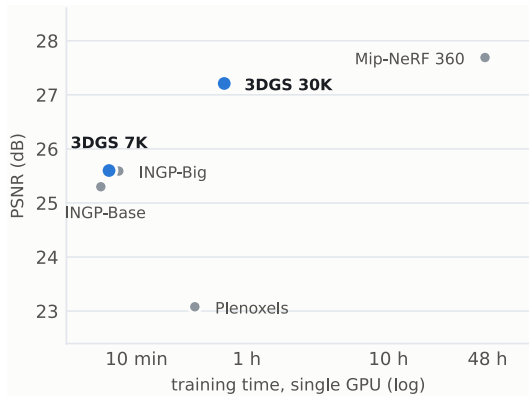


Figure 2 — quality against cost, Mip-NeRF 360. The same row of Table 1, plotted. 3DGS reaches Mip-NeRF 360's quality band in about forty minutes rather than two days; its 7 K checkpoint already matches Instant-NGP. Both 3DGS points are free by-products of one run, since `full_eval.py` renders at 7 000 and 30 000 (F13).

3DGS

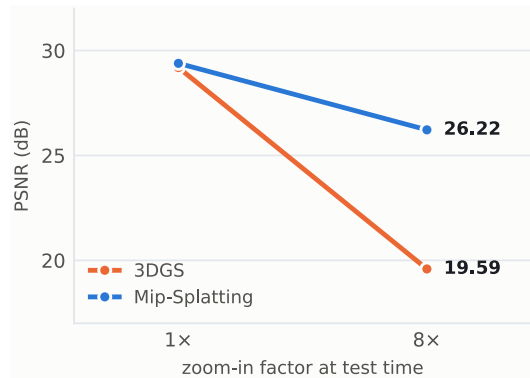


Figure 3 — the same failure on real scenes. Mip-NeRF 360, trained at one scale and tested zoomed in. The 6.63 dB gap at 8× is the real-scene counterpart of Figure 1 left. Do not cross-compare the 1× value of 29.19 with Table 4.1's 27.21 — different protocol, different resolution basis.

MIP · PAPER

4.2 · Mip-Splatting, Blender — the two protocols, as numbers MIP · PAPER

TEST SCALE	STMT 3DGS	STMT MIP-S.	Δ	MTMT 3DGS	MTMT MIP-S.	Δ
Full res	33.33	33.36	+0.03	28.79	32.81	+4.02
1/2	26.95	34.00	+7.05	30.66	34.49	+3.83
1/4	21.38	31.85	+10.47	31.64	35.45	+3.81
1/8	17.69	28.67	+10.98	27.98	35.50	+7.52

Internal consistency check worth stating in the viva: STMT full-res 3DGS is 33.33 here against 33.32 in §4.1 — the same baseline, measured independently by two groups, agreeing to 0.01 dB. That is the empirical justification for the 0.20 dB L1 tolerance.

4.3 · Independent third-party reference — Mip-NeRF 360 NB

METHOD (NERFBASELINES ID)	PSNR	SSIM	LPIPS	TIME
gsplat	27.41	0.815	0.256	29m19s
taming-3dgs	27.22	0.793	0.305	5m59s

3dgt	27.04	0.812	0.252	34m34s
2d-gaussian-splatting	26.81	0.796	0.297	31m10s
instant-ngp	25.51	0.684	0.398	3m54s

The 0.6 dB spread between faithful splatting implementations on one benchmark is the honest answer to "why isn't your number exactly 27.21".

4.4 • Constants confirmed in source

QUANTITY	VALUE	CONFIRMED IN
Blender scenes	ship drums ficus hotdog lego materials mic chair	MIP·SRC F12
Training iterations	30000 ; render also at 7000	MIP·SRC GS·SRC F13, F12
Blender flags	--eval -- white_background	MIP·SRC F12
Mip-Splatting 2D filter	--kernel_size 0.1 (3DGS uses a fixed 0.3f)	MIP·SRC HB
3D smoothing filter variance s	0.2 , recomputed every 100 iterations	MIP·PAPER HB
LPIPS backbone	VGG in both repos — lpips.LPIPS(net='vgg') / net_type='vgg'	MIP·SRC GS·SRC F8
Mip-NeRF 360 resolution	outdoor -i images_4 , indoor -i images_2	GS·SRC F13
Test split	COLMAP scenes: every 8th photo. Blender: the dataset's own test split	MIP·SRC F7 — 1lffhold=8 is in readColmapSceneInfo only

PART II • EXECUTING IT ON FREE COMPUTE

05 The compute envelope, and what it forbids

PLATFORM	HARDWARE	CONSTRAINT THAT SHAPES THE PLAN
Kaggle Notebooks primary	2 × T4 (16 GB), CC 7.5 P100 ineligible — F9	30 h weekly quota · 12 h maximum GPU session · ~20 GB session storage SECONDARY — read the true values off the notebook sidebar in session 1 (F15). The plan is insensitive to them because every run checkpoints; only the calendar depends on them. With F11's dispatcher, R1 is roughly 5–8 h wall-clock, so it fits one session . Input mounts read-only from a private Kaggle Dataset — the reason to prefer Kaggle over Colab. The two T4s are used by the authors' own dispatcher (F11).

Google Colab (free) 1 × T4, not guaranteed overflow Shorter, pre-emptible sessions; no persistent input mount without Drive. Use for short instrumented jobs and figure generation, never for 30 K training runs.

The binding constraint. 16 GB against a stated requirement of 24 GB `GS·SRC`, with peak training memory reported above 20 GB on large scenes `3DGS`. This removes an experiment; it is not a preference to tune around.

EXPERIMENT	FITS 16 GB?	CONSEQUENCE
Blender synthetic, both protocols	YES	Full L1 claim available. 800×800, no COLMAP stage.
Deep Blending — drjohnson, playroom	LIKELY	Indoor, bounded. Confirm at R0; published model size 676 MB.
Tanks & Temples — truck, train	LIKELY	Repository default resolution, as <code>full_eval.py</code> uses.
Mip-NeRF 360 indoor — room, counter, kitchen, bonsai	LIKELY	<code>-i images_2</code> . Most probable OOM candidate: <code>counter</code> .
Mip-NeRF 360 outdoor — bicycle, flowers, garden, stump, treehill	NO	Removed from the L1 claim. Run at <code>-i images_8</code> , reported as L2 with the resolution in the caption. Never placed in a column headed by a published average.

THIS IS R-1 FROM THE HANDBOOK, NOW REALISED

Handbook §13 lists "GPU unavailable, or under 24 GB" as a HIGH risk with the response "Blender synthetic plus a few real scenes at half resolution ... the reduction test, the stress suite and the diagnostic are resolution-independent, so the argument survives; only the scene count shrinks." That response is now the plan, adopted deliberately rather than under pressure.

06 R1 — the primary target, in full

Specified in full because F1 showed it is the one rung with no shipped recipe, F2 settled how it must be built, and F5 showed the shipped pipeline cannot score it.

Data

```
# in the mip-splatting repo, from the standard NeRF synthetic release
python convert_blender_data.py --blender_dir nerf_synthetic/ --out_dir multi-scale
# writes images_train/ images_val/ images_test/ metadata.json (NOT transforms_*.json) — F2
# "single-scale train" is the load_allres=False default, not a separate directory — F4
```

Arm A — Mip-Splatting `AS SHIPPED`

```
pip install gputil lpips # dispatcher F11, metrics F8
python scripts/run_nerf_synthetic_stmt.py
# per scene, verbatim from the script:
# train.py -s multi-scale/{scene} -m {out}/{scene} --eval --white_background --kernel_size 0.1
# render.py -m {out}/{scene} --skip_train
# metrics.py -m {out}/{scene}
```

Arm B — 3DGS TWO LINES, PROVABLY EXACT

Same codebase, same script, two flags added — `--kernel_size 0.3 --disable_3D_filter` — backed by one diff on a branch named `arm-b-3dgs-baseline` :

```
# arguments/__init__.py, in ModelParams.__init__, beside self._kernel_size = 0.1
self.disable_3D_filter = False

# scene/gaussian_model.py, last line of compute_3D_filter
if getattr(self, "_disable_3D_filter", False):
    self.filter_3D = torch.zeros_like(filter_3D[...], None)
else:
    self.filter_3D = filter_3D[...], None]
```

Why this is 3DGS exactly, not an approximation. By F3 the filter enters only through two properties. With `filter_3D = 0`, `get_scaling_with_3D_filter` returns $\sqrt{(s^2+0)} = s$, and in `get_opacity_with_3D_filter` the determinants coincide so `coef = 1` and opacity passes through unchanged. Both become the identity. With `--kernel_size 0.3` matching 3DGS's fixed `0.3f` 2D dilation, the model is 3DGS. This is the exact-reduction unit test Handbook §09 scheduled for W5, available in week 1 at no cost.

Scoring — the step that is not shipped REQUIRED

By F5, `metrics.py` blends all four scales into one number. The four columns are recovered from `per_view.json` without re-rendering:

```
python tools/split_by_scale.py --model-root <out> --data-root multi-scale \
--method ours_30000 --csv results/runs.csv --tag method=mip-splatting,arm=A,train_scale=1x
```

The mapping it relies on, so it can be checked rather than trusted: `readMultiScale` iterates `metadata.json["test"]["file_path"]` in order, appending one camera per entry; `render_set` then does `for idx, view in enumerate(views)` and saves `'{0:05d}'.format(idx)`. Order is preserved end to end, so `render {idx:05d}.png` corresponds to `file_path[idx]`, whose `_d{jj}` suffix gives the scale. Pre-flight check 9 verifies this by reconstructing `results.json`'s single number as the count-weighted mean of the four groups.

GATE G2 — THE EXIT CRITERION FOR R1

3DGS falls by ≥ 13 dB from full resolution to $\frac{1}{8}$; Mip-Splatting falls by ≤ 6 dB; the gap at $\frac{1}{8}$ exceeds 9 dB (published: 15.64, 4.69, 10.98). If it does not, *stop* — the harness is wrong, not the paper. Diagnose in this order: (a) the splitter's index mapping, via pre-flight check 9; (b) whether arm B's `filter_3D` is actually zero at step 30 000 — print its maximum; (c) `--white_background`. Do not proceed to method work with an untrusted harness.

07 The full ladder and its budget

RUNG	RUN	GPU-H	EXIT CRITERION
		EST	
R0	Smoke. lego , 7 K iterations, both arms, end to end including metrics — plus the whole of \$13.	0.5 — 1	Both arms train, render and score without hand-editing; \$13 fully green. Measured iterations/second replaces every EST .

R1	Blender STMT (§06). 8 scenes × 2 arms × 30 K, tested at 1×, ½, ¼, ⅛.	10 – 16	Gate G2 above.
R2	Blender MTMT. run_nerf_synthetic_mmt.py for arm A; the matching arm B.	14 – 22	Mip-Splatting ahead at every scale, gap widening at ½. L1 if within 0.20 dB of §4.2.
R3	Real scenes that fit. 360 indoor ×4 + T&T ×2 + DB ×2, both methods, settings per F13. nerfbaselines usable here.	20 – 30	Per-scene table, L1 only where resolution matches the paper. DB average within 0.3 dB of §4.41.
R5	Seed spread. 3 seeds × 2 scenes × 2 methods.	4 – 8	The σ that makes parity testable. Not optional.
R4	Outdoor, reduced. 5 scenes at -i images_8. The only authorised deviation.	12 – 20	L2 only, own table, resolution in the caption. Cut first if quota binds.

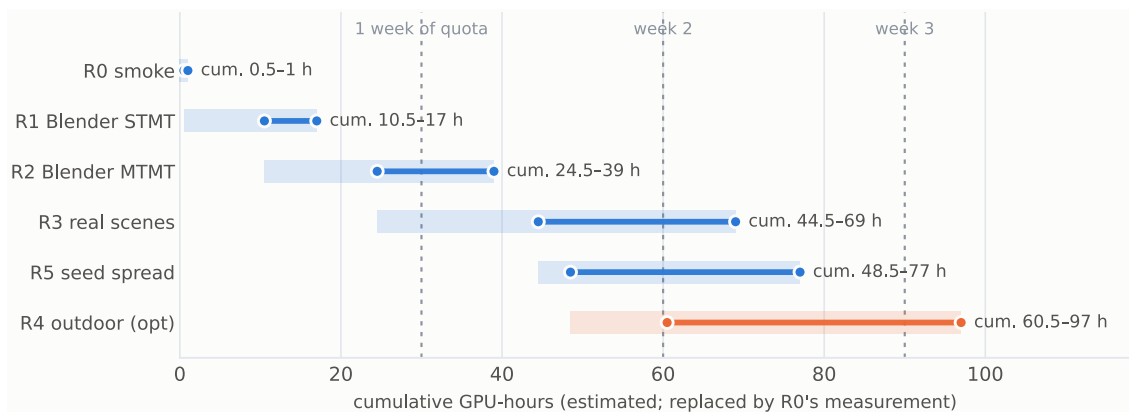


Figure 4 — cumulative budget against a 30 h weekly quota. Bands span the optimistic and pessimistic cumulative positions. R0–R3 plus R5 lands at **48.5–77 GPU-hours** for one seed of the main tables. Two readings matter: the ladder is ordered so that an exhausted week always ends on a completed rung, and — because the authors' dispatcher keeps both T4s busy (F11) and quota is metered in session wall-clock — the *elapsed* quota for R1 and R2 is roughly half the GPU-hours shown. Estimates are replaced by R0's measurement.

Basis for the estimates. 3DGS reports 41m33s mean on Mip-NeRF 360 and 27–36 min on T&T / DB on one A6000 3DGS; nerfbaselines reports ≈29 min for a comparable implementation on a modern card NB. A T4 has roughly 2.4× lower memory bandwidth than an A6000, and 3DGS states about 80 % of training time is spent in Python rather than CUDA, so the slowdown should be milder than a raw FLOP ratio suggests. A 3–4× factor is assumed. **R0's real purpose is to replace this paragraph with a measurement.**

08 Toolchain, data, licence

Repositories — one clone, both arms

```
git clone --recursive https://github.com/autonomousvision/mip-splatting
export TORCH_CUDA_ARCH_LIST="7.5" # T4 = sm_75. P100 (sm_60) is ineligible — F9
pip install ninja gputil lpips
pip install submodules/diff-gaussian-rasterization submodules/simple-knn
# PIN AND RECORD the repo commit and both rasteriser commits (HB §12).
```

```
# The README pins python=3.8 / torch==1.12.1+cu113 — do NOT recreate that on a 2026 image;  
# build the extensions against the installed torch and record what you built against.
```

The original 3DGS repository is **not cloned for the critical path**: by F2 it cannot read the multi-scale data, and by F3 arm B reduces to it exactly inside this codebase. Clone it after R3 only, to cross-check the COLMAP scenes it *can* read. nerfbaselines (`pip install nerfbaselines`) carries both methods as official wrappers but exposes only standard dataset protocols, so it cannot run R1 or R2 (F14); keep it as the independent reference behind §4.3.

GSPLAT IS A CROSS-CHECK, NOT A SUBSTITUTE

`gsplat`'s `rasterize_mode="antialiased"` applies the view-dependent opacity compensation $p = \sqrt{(\det \Sigma / \det(\Sigma + \epsilon I))}$, attributed to Mip-Splatting. That is the **2D** Mip filter's compensation term. It is *not* the **3D** smoothing filter — the per-Gaussian `filter_3D` of shape `[N, 1]` holding $0.2 \cdot (d_{\min}/f_{\max})^2$, which is the exact object B1 replaces `HB`. The same caution applies to 3DGS's own `--antialiasing` flag (F2). State in the thesis which of the two filters each implementation contains.

DATASET	SOURCE	NOTES
Blender synthetic (8)	NeRF authors' release, <code>nerf_synthetic.zip</code> ; also <code>external://blender/<scene></code> <code>NB</code>	Smallest and fastest; the R1/R2 target. Multi-scale variant via <code>convert_blender_data.py</code> .
Tanks & Temples + Deep Blending	<code>repo-sam.inria.fr/fungraph/3d-gaussian-splatting/datasets/input/tandt_db.zip</code> <code>GS·SRC</code>	Ships COLMAP poses; no SfM stage.
Mip-NeRF 360 (9)	Barron et al. project page — main release plus an extra-scenes archive for <code>flowers</code> and <code>treehill</code> <code>GS·SRC</code>	Largest download. Only the 4 indoor scenes are needed for R3.

RESIDENCY RULE

Download once, then publish the prepared data — including the generated `multi-scale/` `tree` — as a **private Kaggle Dataset**, mounting read-only at `/kaggle/input/` at no cost in quota. Re-downloading a multi-gigabyte archive inside a metered GPU session is the second most common way to waste this budget, after fighting the toolchain. Checkpoints, `results/runs.csv` and figures go out to a private Hugging Face Hub repository *before* the session ends — `/kaggle/working` does not survive.

Licence `GS·SRC`. 3DGS is released under the Gaussian-Splatting License of Inria and the Max Planck Institut für Informatik: royalty-free for "*research and/or evaluation purposes only*", requiring that redistribution carry a complete copy of the licence and retain all notices, with no commercial use without prior consent (`stip-sophia.transfert@inria.fr`). Mip-Splatting's README states it follows the 3DGS licence. A BTP is squarely within permitted use; the Spring code release must ship the licence text, and the constraint belongs in the thesis rather than being discovered at submission.

PART III · TURNING THE REPRODUCTION INTO THE FIRST EXPERIMENT

09 Three instruments to attach to the baseline runs

All three are read-only with respect to the optimiser: nothing below changes a training decision, so the reproduction remains a reproduction. Together they answer, before any method exists, whether the method's premise holds.

I-1 • Floor occupancy KILL-SHOT

Mip-Splatting stores one scalar per Gaussian, `filter_3D` of shape $[N,1]$, computed as $\text{distance} / \text{focal_length} * (0.2 ** 0.5)$ with the minimum depth over all cameras and the maximum focal length taken independently HB. Log its full distribution at the end of training, per scene, with each Gaussian's distance to its nearest camera and its final scale vector.

What it decides. B1's estimation term is visible only where $s^2/\lambda_i > f_k^2$. If the floor sits at or above the cap on nearly every Gaussian in every scene, B1 collapses to Mip-Splatting universally and the project must pivot. **The cheapest available falsification of the central idea, and a by-product of a run already scheduled.**

I-2 • Λ conditioning profile

On a *trained, unmodified* 3DGS checkpoint, accumulate $\Lambda_k = \sum_n w_{nk}(J_n W_n)^T \Sigma_{\text{pix}}^{-1}(J_n W_n)$ and record λ_3/λ_1 per Gaussian and the angle between the smallest eigenvector and the mean viewing direction. Pure PyTorch, no CUDA.

What it decides. The thesis asserts the resolvable-detail limit is directional — a statement about Λ 's conditioning, measurable on existing checkpoints. Predicted: λ_3/λ_1 near 1 on well-conditioned orbits, far from 1 on the low-parallax and one-sided protocols of Handbook §11. That plot turns Handbook §07's regime table from hopes into supported hypotheses, and is the strongest possible opening for the W8 seminar.

I-3 • Run-to-run spread

From R5: per-scene mean and standard deviation across three seeds, both baselines. Converts "parity" into a quantity. Report parity as "*within $k \sigma$ of the self-run baseline*" and the claim is unassailable; report it as "about the same" and a panel will press on it.

WHY THIS REFRAMES THE REPORT

A reproduction that only regenerates published tables produces one sentence for the thesis. A reproduction carrying I-1, I-2 and I-3 produces the error bar the headline prediction needs, a falsification test that costs nothing extra, and the figure that motivates the whole approach — all before week 5. The extra cost is instrumentation code, not GPU time.

10 Decision gates

GATE	AFTER	QUESTION	BRANCHES
------	-------	----------	----------

G0	§01, closed	Which codebase runs arm B?	Closed by F2 before deployment. The original 3DGS repository cannot load the multi-scale data at all, so both arms run in the Mip-Splatting codebase and arm B is the two-line reduction of §06. This was v2's one open blocking question; it is now a stated decision, and the only thing left is the supervisor's sign-off on the diff.
G1	R0	Do both arms build and run end-to-end on free compute?	Yes → proceed; replace every EST with measurement. No → 3 days on toolchain repair, hard-stopped, then escalate: departmental GPU or institute HPC, requested with R0's failure log attached. Do not spend a second week.
G2	R1	Does the 1/8-scale gap exceed 9 dB?	Yes → baselines trusted. No → <i>stop</i> . Diagnose in the order given in §06. Do not proceed to method work with an untrusted harness.
G3	I-1	Does the estimation term have room above the inherited floor?	Yes → B1 proceeds. Narrow → narrow the claim now to mixed-focal and one-sided regimes — smaller, still defensible. No → pivot to B2 (SH identifiability), untouched by this, or the Handbook §16 graph reserve. Cost of pivoting here: two weeks. Cost in March: the project.
G4	I-2	Is Λ near-isotropic on benchmarks and anisotropic on the stress protocols?	Yes → the predicted result; parity is mechanistically explained rather than hoped for. Anisotropic everywhere → <i>better</i> : expect a gain on standard benchmarks too. Update Handbook §07 <i>before</i> the runs, so it is a prediction and not a retrofit.
G5	Week 2, in parallel	Handbook §08 due-diligence: has anyone already made the 3D filter anisotropic or per-direction?	Unchanged, and still the only thing that can invalidate the project. It runs on <i>literature</i> time, not GPU time — which is exactly why it belongs in the compute-bound weeks.

11 Risk register — delta against Handbook §13

ID	RISK	STATUS	RESPONSE
R-1	GPU unavailable or under 24 GB	REALISED	The operating condition, not a risk. §05 and §07 are the response; the handbook's own mitigation applies unchanged.
R-5	Reproduction does not match published numbers	RESHAPED	Partly guaranteed by §03. The residual risk is <i>unexplained</i> mismatch; the three-level scheme plus recorded commits and LPIPS backbone is what separates the two.
R-10	Weekly quota exhausted mid-table	MED	Plan at 60 % utilisation. Strict ladder order, so an exhausted week ends on a completed rung. F11's 2× parallelism buys back most of the margin.

R-11	Session pre-empted mid-run	HIGH	Non-negotiable: <code>--checkpoint_iterations</code> and <code>--start_checkpoint</code> on every run; checkpoints pushed off-session before the run ends. A 30 K run that cannot resume will eventually be lost.
R-14	The per-scale split is silently misaligned	NEW · HIGH	Replaces v2's arm-B risk, which F2 eliminated. If render indices and <code>metadata.json</code> order ever diverge, every column is wrong while every number looks plausible. Pre-flight check 9 detects it by reconstructing <code>results.json</code> 's blended average from the four groups; the 33.3 full-res anchor catches the rest.
R-15	A metrics failure is read as a bad result	NEW · MED	F6's bare <code>except: .Assert results.json exists</code> and is non-empty after every metrics call; never infer a number from console output.
R-16	LPIPS backbone drift	NEW · MED	F8. Pin <code>net_type='vgg'</code> ; the backbone is a column in the results schema, not an assumption.
R-13	Free-tier terms change mid-project	LOW	Keep the ladder platform-agnostic: one <code>make target</code> per table, driven by config, so the same commands run on Kaggle, Colab or a departmental machine.

12 Revised Autumn schedule

Handbook §09 assumed one 24 GB GPU and a W2 exit of "within 0.2 dB of published on three scenes" — not attainable on 16 GB for real scenes, and replaced by §03. Sequencing and exit discipline are otherwise unchanged; only the compute-bound weeks move.

WEEK	WORK	EXIT CRITERION
W1	Accounts and quotas confirmed. Data downloaded, multi-scale/ built, republished as private datasets. §13 pre-flight , then R0 . Literature alerts set; Handbook §08 due-diligence begun; BTP administrative facts collected.	G0 and G1 passed. Arm B decided in writing; measured iterations/second recorded.
W2	R1 , both arms. Instrument I-1 attached to the Mip-Splatting arm. §08 due-diligence continues while runs are queued.	G2 passed. Figure 1 redrawn from our own data. Floor histograms for 8 scenes.
W3	R2 and R5 . I-2 implemented against trained checkpoints — pure PyTorch. Derivation work proceeds in parallel; it needs no GPU.	G3, G4. σ table. Λ conditioning plots, 2 scenes $\times$ 4 protocols.
W4	R3 . Anisotropic-capture stress suite built from the same images (Handbook §11) — camera-subset construction is free.	Real-scene baseline table complete for the scenes that fit; four protocols versioned.
W5–W6	Rejoins Handbook §09 W4–W5: implement Λ accumulation, the exact-reduction unit test, wire Σ_{filt} into the covariance path.	Reduction test passes to 1e-6; trains to 30 K without divergence.
W7–W13	Handbook §09 W6–W13 unchanged, with R4 as optional overflow and Experiment A cut per its stated cut rule unless quota is unexpectedly generous.	Unchanged. The W8 seminar now opens on the I-2 figure rather than a promise.

Net effect: reproduction takes three to four weeks rather than two, and buys three results the original plan did not have. Derivation, stress-suite construction and §08 due-diligence are all zero-GPU, so they fill exactly the hours the quota forces you to spend waiting — the one genuine advantage of a compute-limited schedule.

13 Pre-flight checklist — the deployment gate

Every item is a *verification*, not a run. All twelve must be green before R1. Gate G0 no longer appears — F2 closed it — so this list is now assertions rather than open questions.

#	CHECK	PASS CONDITION
1	Accelerator	<code>torch.cuda.get_device_capability() == (7,5)</code> . Abort on P100. F9
2	Installs	<code>ninja</code> , <code>gputil</code> , <code>lpips</code> , then both submodules with <code>TORCH_CUDA_ARCH_LIST="7.5"</code> . F8 F11
3	Commits pinned	Repository HEAD and both submodule commits written into <code>results/runs.csv</code> from row 1.
4	Arm B diff applied	Exactly as §06, on branch <code>arm-b-3dgs-baseline</code> , and the diff printed. F3
5	Arm B unit test	<code>lego</code> , filter disabled, <code>--kernel_size 0.3</code> → full-res PSNR within 0.5 dB of 33.3 .
6	Arm A sanity	<code>lego</code> as shipped → full-res PSNR within 0.5 dB of 33.4 .
7	Multi-scale data	Four distinct <code>_d{j}</code> groups in the test split; the training split under the <code>load_allres=False</code> default contains only <code>d0</code> files. F4
8	LPIPS backbone	Assert VGG at import; recorded as a CSV column. F8
9	Splitter alignment	<code>split_by_scale.py</code> reproduces <code>results.json</code> 's single number as the count-weighted mean of its four groups. This is what proves the index mapping. F5
10	Metrics failure detection	<code>results.json</code> asserted present and non-empty after every metrics call. F6
11	Resume	Kill at 2 K, restart from checkpoint, reach 30 K. R-11
12	Off-session persistence	Dummy checkpoint and CSV retrievable from a fresh session. F16

CHECKS 5 AND 6 DESERVE THEIR OWN LINE

Two independent groups measured 3DGS at full resolution on Blender and got 33.32 and 33.33. Arm B must land there, and arm A near 33.4. These two numbers, visible after one scene each, are the cheapest possible test that the reduction of §06 is exact and that the data is being read correctly — before the other thirty runs are spent.

14 Open items and questions for the supervisor

Everything still unknown — two items

- **Kaggle's true weekly quota and session cap.** Secondary sources give 30 h/week and a 12 h session (F15). Read them off the notebook sidebar in session 1. The ladder does not depend on them; only the calendar in §12 does.
- **Whether counter and drjohnson fit 16 GB** at their `full_eval.py` resolutions. Unknowable without running. R3's exit rule already covers it: record the OOM, do not work around it. Everything marked **LIKELY** in §05 is a prediction of this kind.

That is the complete list. Every other question this report raised across v1 and v2 has been closed against source in §01.

Decisions to ask for

1. **Is departmental or institute GPU time available?** The highest-leverage question. A 24 GB card converts §03's L2 claims into L1 and restores five outdoor scenes. Ask before improvising around its absence, and attach R0's measured throughput as the evidence.
2. **Agree the parity protocol now**, per Handbook §14. With §03 and I-3 in hand this is concrete: parity means within $k\sigma$ of the self-run baseline, and k should be fixed before the number exists.
3. **Confirm L2 + L3 reporting is acceptable to the panel.** Standard in this literature, and 3DGS does it — but an EE panel that does not read graphics should be told in the thesis rather than discover it in the viva.
4. **Approve arm B once §13 check 1 is answered.** If the readers differ, B-ii means one recorded diff to a published codebase; that is worth a supervisor's explicit sign-off rather than a student's unilateral choice.
5. **Confirm scope**, per Handbook §14: B1 alone is a pass-level contribution, so R4 and Experiment A can be cut without penalty.

ONE-LINE SUMMARY FOR THE WEEKLY ONE-PAGER

Protocol verified against repository source before deployment: ten corrections, four of which would have produced a wrong reproduction rather than a failed one. The primary target is Blender single-scale → multi-scale, whose 11 dB effect survives any hardware difference; its 3DGS arm is not shipped by either repository and is now specified with a written decision gate ahead of it; and three read-only instruments on the baseline runs deliver the parity error bar, the project's cheapest falsification test, and the motivating figure.

15 Appendix — reproducibility artefacts

ARTEFACT	PURPOSE
D:\BTP\CLAUDE-CODE-REPRODUCTION-PROMPT.md	The operating prompt, pasted as the first message of a Claude Code session in the reproduction repository. Encodes §03's levels, §06's two arms, §07's ladder, §09's instruments, §10's gates and §13's pre-flight, with an explicit prohibition on inventing or silently altering any number.

D:\BTP\make_figures.py

Generates every figure in this report. `--mode published` draws the paper's numbers; `--mode measured` redraws them from `results/runs.csv` with the published curve retained as a faint dashed reference, and **refuses to run** if the CSV has no usable rows — so a "measured" figure can never contain a published number.

D:\BTP\split_by_scale.py

Required to produce Table 2 at all (F5). Groups `per_view.json` by test scale using the order-preserving render-index map, emitting the four columns and rows ready for `results/runs.csv`. Pre-flight check 9 validates its alignment against `results.json`.

results/runs.csv

The instruction from the operating prompt worth repeating here: *a cell in the results table is either measured by a logged run or it is empty. There is no third state.*

Scope of verification. Published figures in §04 come from the 3DGS paper as recorded in Handbook §02 and from the Mip-Splatting paper. Findings F1, F2, F11, F12 and F12 come from the complete contents of `autonomousvision/mip-splatting/scripts/` and the full source of `run_nerf_synthetic_stmt.py`; F9, F8, F2 and F13 from `graphdeco-inria/gaussian-splatting's` `README.md`, `metrics.py` and `full_eval.py`; F14 from the nerfbaselines `README` and `leaderboard` — all on `main`, read 6–7 September 2026. Free-tier quota figures and the quota-metering assumption behind F11 are marked unverified and sit on the §13 and §14 lists. Nothing else is asserted.